

Creación y básicos

Función/Método	Qué hace	Ejemplo → Resultado
<code>list(iterable)</code>	Crea lista desde iterable	<code>list('abc') → ['a', 'b', 'c']</code>
<code>len(l)</code>	Longitud	<code>len([1,2,3]) → 3</code>
<code>type(l)</code>	Tipo del objeto	<code>type([1,2]) → <class 'list'></code>
<code>l.copy()</code>	Copia superficial	<code>l=[1,[2]]; l2=l.copy(); l is l2 → False</code>

Indexado y slicing

Sintaxis	Qué hace	Ejemplo → Resultado
<code>l[i]</code>	Elemento en posición <i>i</i>	<code>[10,20,30][1] → 20</code>
<code>l[-1]</code>	Último elemento	<code>[1,2,3][-1] → 3</code>
<code>l[i:j]</code>	Rebanada (mitad abierta)	<code>[0,1,2,3][1:3] → [1,2]</code>
<code>l[:j], l[i:]</code>	Desde inicio / hasta fin	<code>[0,1,2][:2] → [0,1]</code>
<code>l[:k]</code>	Paso	<code>[0,1,2,3][:2] → [0,2]</code>
<code>l[::-1]</code>	Invertida (copia)	<code>[1,2,3][::-1] → [3,2,1]</code>
Asignación por slicing	Reemplaza rango	<code>l=[1,2,3]; l[1:3]=[9] → l=[1,9]</code>

Añadir / quitar elementos

Método	Qué hace	Ejemplo → Resultado
<code>l.append(x)</code>	Añade al final	<code>l=[1]; l.append(2) → l=[1,2]</code>
<code>l.extend(it)</code>	Añade varios	<code>l=[1]; l.extend([2,3]) → l=[1,2,3]</code>
<code>l.insert(i,x)</code>	Inserta en <i>i</i>	<code>l=[1,3]; l.insert(1,2) → l=[1,2,3]</code>
<code>l.remove(x)</code>	Quita primera ocurrencia	<code>l=[1,2,2]; l.remove(2) → l=[1,2]</code>
<code>l.pop(i=-1)</code>	Extrae y devuelve	<code>l=[1,2]; l.pop() → 2, l=[1]</code>
<code>del l[i:j]</code>	Borra por índice/rango	<code>l=[0,1,2]; del l[1:] → l=[0]</code>
<code>l.clear()</code>	Vacía la lista	<code>l=[1,2]; l.clear() → l=[]</code>

Búsqueda y conteo

Método	Qué hace	Ejemplo → Resultado
<code>l.index(x[, start, end])</code>	Índice de <i>x</i> (lanza <code>ValueError</code> si no está)	<code>[10,20,10].index(10) → 0</code>
<code>l.count(x)</code>	Nº de ocurrencias	<code>[1,1,2].count(1) → 2</code>

Método	Qué hace	Ejemplo → Resultado
<code>x in l / x not in l</code>	Pertenencia	<code>2 in [1,2,3] → True</code>

Ordenar e invertir

Método/Función	Qué hace	Ejemplo → Resultado
<code>l.sort(key=None, reverse=False)</code>	Ordena in place	<code>l=[3,1,2]; l.sort() → l=[1,2,3]</code>
<code>sorted(iterable, key=..., reverse=...)</code>	Devuelve nueva lista ordenada	<code>sorted(['a','BB','c'], key=len) → ['a','c','BB']</code>
<code>l.reverse()</code>	Invierte in place	<code>l=[1,2]; l.reverse() → l=[2,1]</code>

Ojo: `l.sort()` y `l.reverse()` devuelven `None`.

Operadores y combinaciones

Operador	Qué hace	Ejemplo → Resultado
<code>l1 + l2</code>	Concatena	<code>[1,2] + [3] → [1,2,3]</code>
<code>l * n</code>	Repite	<code>[0]*3 → [0,0,0]</code>
Desempaquetado	Asigna en varias vars	<code>a,*m,b = [1,2,3,4] → a=1, m=[2,3], b=4</code>

Recorridos y utilidades

Función	Qué hace	Ejemplo → Resultado
<code>enumerate(l, start=1)</code>	Índice + valor	<code>list(enumerate(['a','b'],1)) → [(1,'a'), (2,'b')]</code>
<code>zip(*iterables)</code>	Agrupar por posición	<code>list(zip([1,2], ['a','b'])) → [(1,'a'), (2,'b')]</code>
<code>map(f, it)</code>	Aplica función (iterador)	<code>list(map(str, [1,2])) → ['1','2']</code>
<code>filter(p, it)</code>	Filtra por predicado	<code>list(filter(lambda x:x%2, [1,2,3])) → [1,3]</code>
<code>sum(l[, start])</code>	Suma numérica	<code>sum([1,2,3]) → 6</code>
<code>min(l), max(l)</code>	Mín/Máx	<code>min([3,1,2]) → 1</code>
<code>any(it), all(it)</code>	Algún/Todos verdaderos	<code>all([True,1,0]) → False</code>

List comprehensions (creación transformando/filtrando)

Patrón	Qué hace	Ejemplo → Resultado
<code>[expr for x in it]</code>	Mapea	<code>[x*2 for x in [1,2,3]] → [2,4,6]</code>
<code>[expr for x in it if cond]</code>	Filtra + mapea	<code>[x for x in range(6) if x%2==0] → [0,2,4]</code>
<code>[[...] for fila in ...]</code>	Anidadas	<code>[(i,j) for i in [1,2] for j in [3,4]] → [(1,3), (1,4), (2,3), (2,4)]</code>

Copias y referencias (muy importante)

Técnica	Tipo	Ejemplo → Resultado
<code>l.copy(), l[:], list(l)</code>	Copia superficial	<code>l=[1,[2]]; l2=l[:]; l2[1][0]=9 → 1 cambia → [1,[9]]</code>
<code>copy.deepcopy(l)</code>	Copia profunda	<code>from copy import deepcopy; l2=deepcopy(l) → Cambios no afectan a l</code>

Gotcha: `[[0]*3]*2` crea **filas referenciadas**; modifica una y cambian todas. Usa comprensión: `[[0 for _ in range(3)] for _ in range(2)]`.
